



Pune Vidyarthi Griha's
COLLEGE OF ENGINEERING, TECHNOLOGY
AND MANAGEMENT, PUNE-09



Pune Vidyarthi Griha's College of Engineering, Technology and Management

Department of Artificial Intelligence and Data Science

Subject Name & Code:

417532(B): Business Intelligence

Student Name: Shreya Yadav

Roll No.: 22028088

Batch : B3

Date of Submission: 20-04-2026

ASSIGNMENT:- 5

AIM:-

Perform the data classification algorithm using any Classification algorithm.

PROBLEM STATEMENT:-

To use the Logistic Regression algorithm for supervised data classification. You have a dataset that contains demographic and travel-related information of Titanic passengers such as age, sex, fare, and passenger class. Apply the Logistic Regression algorithm to classify whether a passenger survived or not based on these features. The objective is to build a predictive model that identifies survival outcomes using labeled data and evaluates its performance using appropriate classification metrics.

THEORY:-

Classification

Classification is a **supervised machine learning** technique used to categorize data into predefined classes based on input features. In Business Intelligence (BI), classification is used to make informed decisions such as whether a customer will churn, whether a patient is at risk, or whether a transaction is fraudulent.

Unlike clustering (unsupervised), classification uses labeled training data to predict the category of new data points.

Types of Classification Algorithms

- **Logistic Regression** (linear classifier)
- **Decision Tree**
- **Random Forest**
- **Support Vector Machine (SVM)**
- **Naïve Bayes**
- **K-Nearest Neighbors (KNN)**

Each has its own strengths depending on the dataset and business objective.

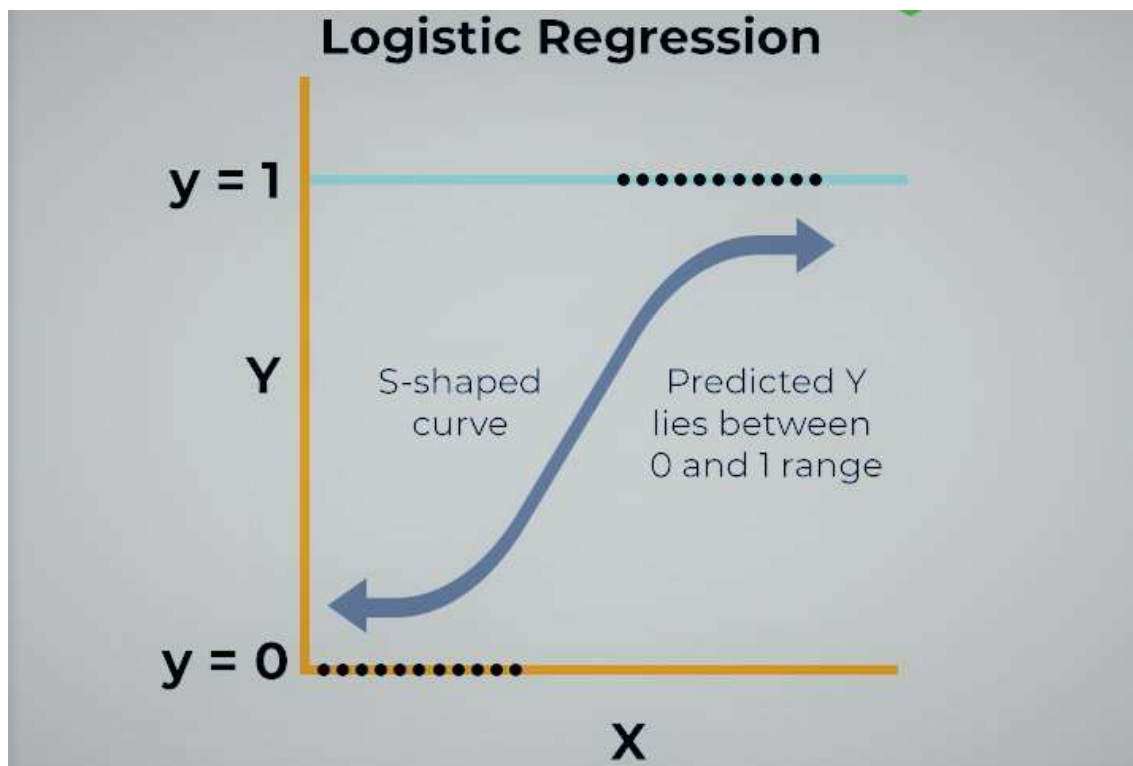
Logistic Regression Classification Algorithm

Logistic Regression is a statistical method commonly used for **binary classification**. It estimates the probability of a binary outcome (0 or 1) based on one or more predictor variables using the **sigmoid function**.

Sigmoid Function:

$$P(y = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}}$$

This output is interpreted as a probability, and a threshold (usually 0.5) is applied to convert it into class labels (0 or 1).



Working of Logistic Regression Algorithm

1. Model Initialization:

- Assume coefficients (β) for each feature.

2. Training the Model:

- Use a dataset with known labels.
- Optimize the coefficients to minimize the error using **maximum likelihood estimation**.

3. Prediction:

- For new data points, calculate the probability of the positive class using the sigmoid function.

4. Classification:

- Assign label 1 if the predicted probability ≥ 0.5 , otherwise 0.

Dataset Used: Titanic Survival Dataset

The Titanic dataset is a classic binary classification dataset used to predict **survival (1) or death (0)** of passengers aboard the Titanic.

Features:

- Age
- Sex
- Pclass (Passenger Class)
- Fare
- Family Size (engineered feature)

Target Variable:

- Survived (0 = No, 1 = Yes)

Data Preprocessing Steps

1. **Missing Value Handling:** Replace missing age values with the median.
2. **Feature Encoding:** Convert categorical features (e.g., Sex) into numerical.
3. **Feature Engineering:** Create new features such as family size from SibSp and Parch.
4. **Feature Scaling:** Standardize the numerical features.
5. **Train-Test Split:** Split data into training and testing sets for evaluation.

Evaluation Metrics for Classification

Metric	Meaning
Accuracy	Percentage of total predictions that are correct.
Precision	Out of all positive predictions, how many were correct.
Recall	Out of all actual positives, how many were correctly predicted.
F1-Score	Harmonic mean of precision and recall.
Confusion Matrix	Table showing TP, TN, FP, FN values.
ROC-AUC Score	Area under the ROC curve; higher values indicate better performance.

Visualization of Classification Results

1. **Correlation Heatmap:** Shows interdependencies among features.
2. **Survival Distribution (Bar Chart):** Highlights imbalance between survived and not.

3. **Survival by Gender (Bar Plot):** Helps understand gender influence on survival.
4. **Age vs Survival (Box Plot):** Insights into which age groups had higher survival.
5. **Fare vs Survival (Violin Plot):** Distribution of fare paid by survivors vs non-survivors.
6. **Pclass vs Survival (Stacked Bar Chart):** Shows class-wise survival percentage.

Applications of Logistic Regression in Business Intelligence

1. **Customer Retention:**
 - Predicting whether a customer is likely to churn.
2. **Credit Scoring:**
 - Determining the probability of loan default.
3. **Medical Diagnosis:**
 - Predicting the presence/absence of disease.
4. **Marketing Response:**
 - Classifying whether a user will click on an ad or not.
5. **Fraud Detection:**
 - Flagging suspicious transactions.

Advantages of Logistic Regression

- Simple and interpretable
- Fast training on large datasets
- Probabilistic output
- Good baseline for binary classification tasks

Limitations of Logistic Regression

- Assumes linear decision boundary
- Sensitive to outliers
- Not suitable for complex nonlinear relationships
- Requires feature scaling for better performance

CONCLUSION

Logistic Regression is a powerful binary classification algorithm, especially useful in BI applications like risk assessment, healthcare analytics, and marketing. When applied to the Titanic dataset, it accurately predicts survival based on multiple passenger attributes. With proper preprocessing, feature engineering, and evaluation, Logistic Regression provides valuable insights for data-driven business decisions.

Code:

The screenshot shows a Jupyter Notebook interface with a light blue header bar. The title bar reads "Jupyter Assignment_5_BI" followed by "Last Checkpoint: an hour ago (autosaved)". On the right, there is a Jupyter logo and a "Logout" button. Below the header is a menu bar with "File", "Edit", "View", "Insert", "Cell", "Kernel", "Widgets", and "Help". To the right of the menu bar are buttons for "Trusted" and "Python 3 (ipykernel)". Below the menu bar is a toolbar with icons for saving, undo, redo, running, and other notebook functions. The main area contains three code cells. The first cell, labeled "In [1]:", contains code to import pandas, numpy, seaborn, matplotlib.pyplot, and sklearn modules. The second cell, labeled "In [2]:", contains code to read a CSV file named "titanic.csv" into a DataFrame. The third cell, labeled "In [3]:", contains code to print the first five rows of the DataFrame. The output of the third cell is displayed below the code, showing two tables. The first table shows the first five rows of the DataFrame with columns PassengerId, Survived, Pclass, and Name. The second table shows the same rows with columns Name, Sex, Age, SibSp, and Parch.

```
In [1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, roc_curve, accuracy_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
In [2]: df = pd.read_csv("titanic.csv")
```

```
In [3]: print(df.head())
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp
0	1	0	3			
1	2	1	1			
2	3	1	3			
3	4	1	1			
4	5	0	3			

	Name	Sex	Age	SibSp
0	Braund, Mr. Owen Harris	male	22.0	1
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1
2	Heikkinen, Miss. Laina	female	26.0	0
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1
4	Allen, Mr. William Henry	male	35.0	0

Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN
1	0	PC 17599	71.2833	C85
2	0	STON/O2. 3101282	7.9250	NaN
3	0	113803	53.1000	C123
4	0	373450	8.0500	NaN

jupyter Assignment_5_BI Last Checkpoint: an hour ago (autosaved) Python 3 (ipykernel)

```
In [4]: print(df.isnull().sum())
```

PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

```
In [5]: # Drop irrelevant features
df.drop(columns=['Name', 'PassengerId', 'Cabin', 'Ticket'], inplace=True)
```

```
In [6]: # Fill missing Age with median, Embarked with mode
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
```

```
In [7]: print(df.isnull().sum())
```

Survived	0
Pclass	0
Sex	0
Age	0
SibSp	0
Parch	0
Fare	0
Embarked	0

dtype: int64

jupyter Assignment_5_BI Last Checkpoint: an hour ago (autosaved) Python 3 (ipykernel)

```
In [8]: # Encode categorical variables
df['Sex'] = LabelEncoder().fit_transform(df['Sex']) # male:0, female:1
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True)
```

```
In [9]: # Create family size feature
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1

# Drop Parch and SibSp
df.drop(columns=['SibSp', 'Parch'], inplace=True)
```

```
In [10]: df.head()
```

```
Out[10]:
```

	Survived	Pclass	Sex	Age	Fare	Embarked_Q	Embarked_S	FamilySize	
0	0	0	3	1	22.0	7.2500	False	True	2
1	1	1	1	0	38.0	71.2833	False	False	2
2	1	1	3	0	26.0	7.9250	False	True	1
3	1	1	1	0	35.0	53.1000	False	True	2
4	0	3	1	35.0	8.0500	False	True	True	1

```
In [11]: #Feature Scaling
scaler = StandardScaler()
scaled_features = scaler.fit_transform(df.drop('Survived', axis=1))
x = pd.DataFrame(scaled_features, columns=df.drop('Survived', axis=1).columns)
y = df['Survived']
```

```
In [12]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42)
```

```
In [13]: model = LogisticRegression()
        params = {
            'C': [0.01, 0.1, 1, 10],
            'solver': ['liblinear', 'lbfgs']
        }
        grid = GridSearchCV(model, param_grid=params, cv=5, scoring='accuracy')
        grid.fit(X_train, y_train)

        best_model = grid.best_estimator_
```

```
In [14]: y_pred = best_model.predict(X_test)

        print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
        print("\nClassification Report:\n", classification_report(y_test, y_pred))
        print("Accuracy:", accuracy_score(y_test, y_pred))
```

Confusion Matrix:

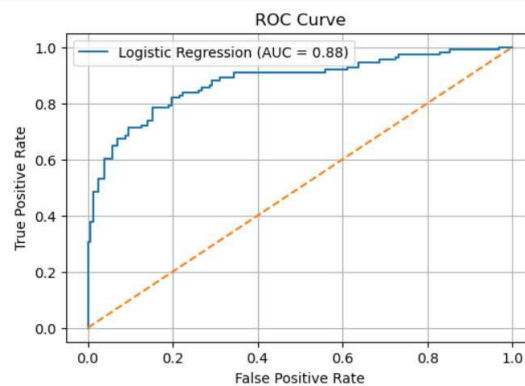
```
[[136  21]
 [ 31  80]]
```

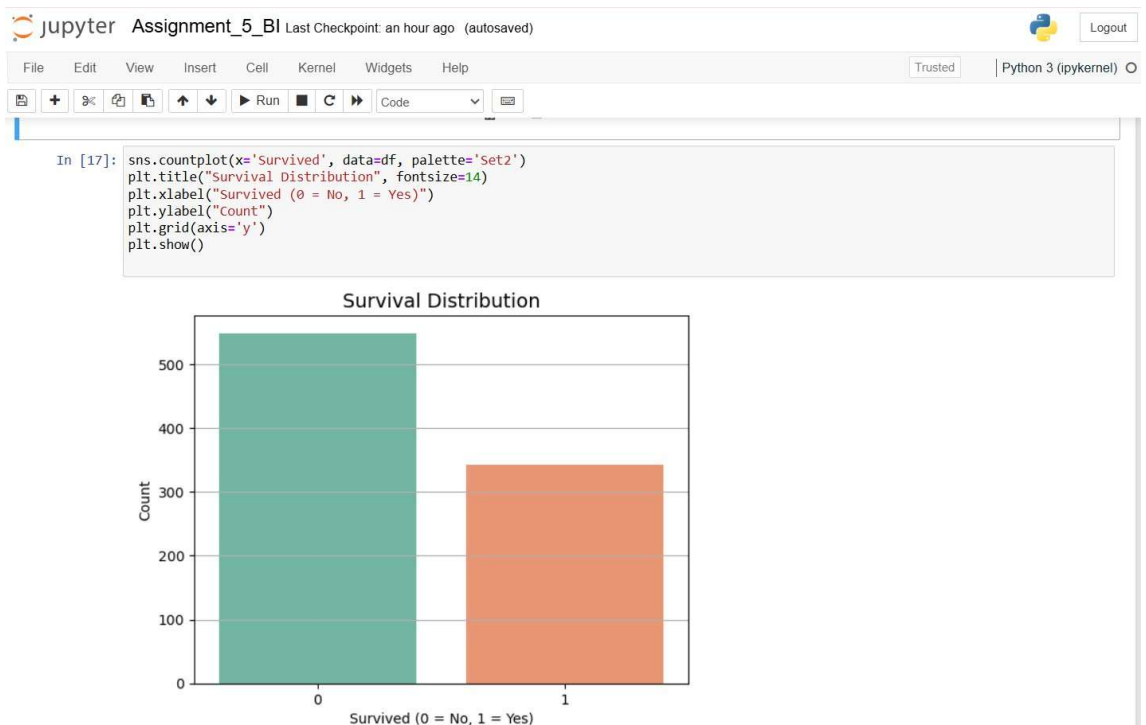
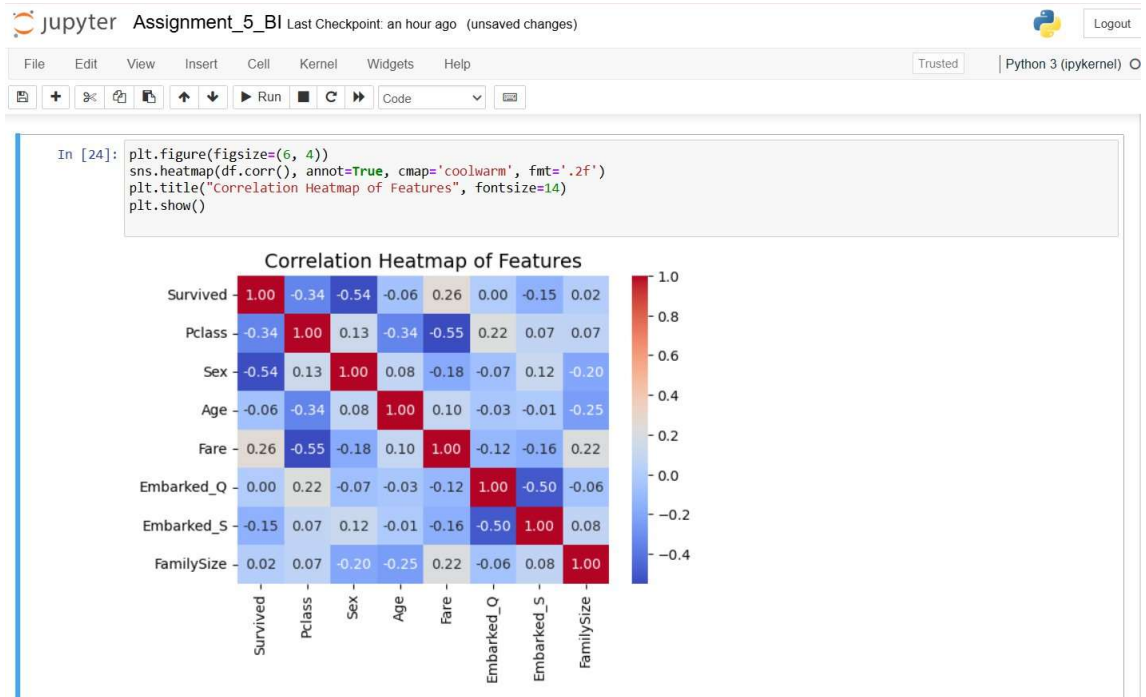
Classification Report:

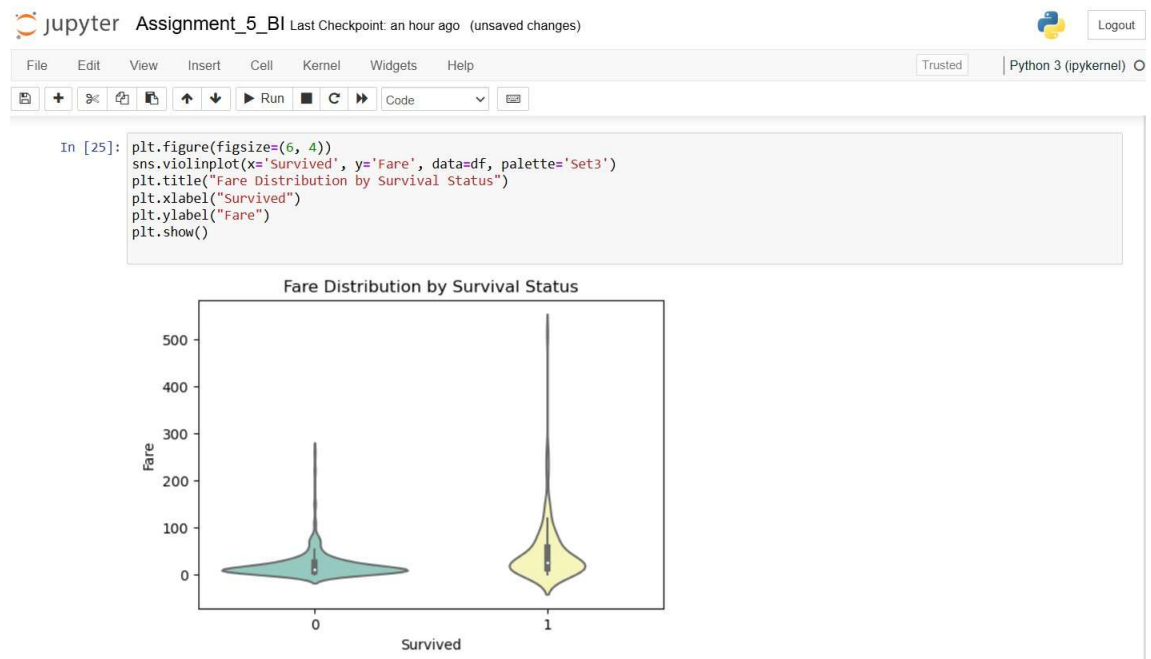
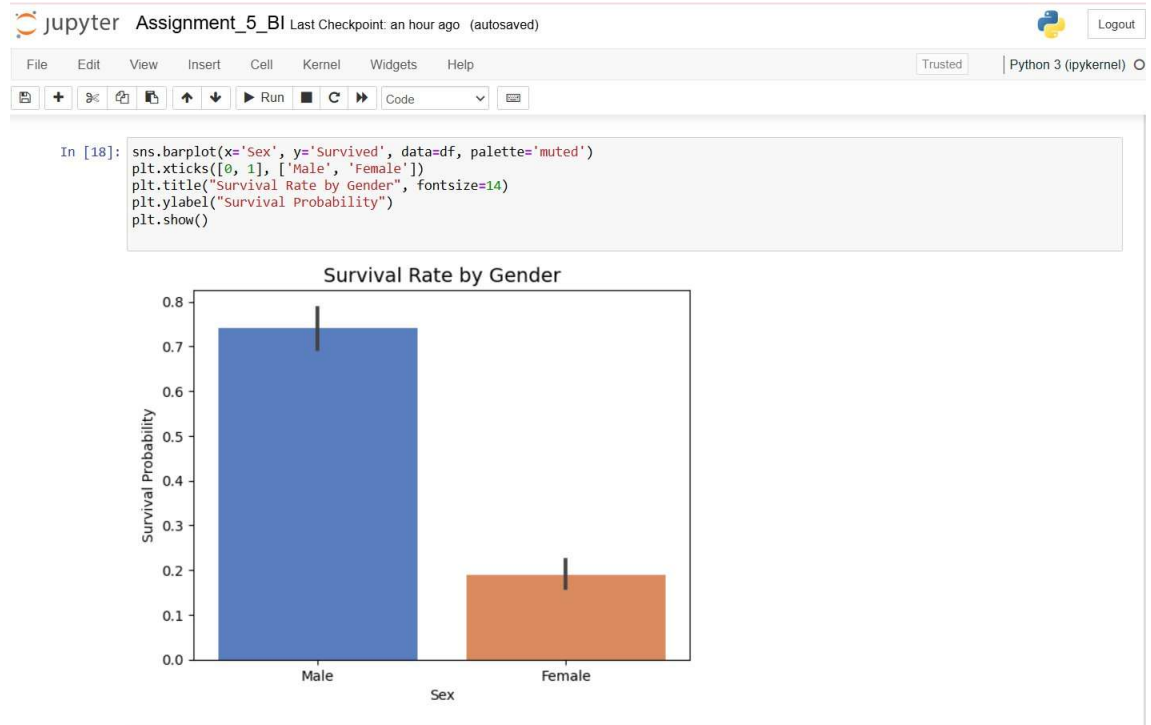
	precision	recall	f1-score	support
0	0.81	0.87	0.84	157
1	0.79	0.72	0.75	111
accuracy			0.81	268
macro avg	0.80	0.79	0.80	268
weighted avg	0.81	0.81	0.80	268

Accuracy: 0.8059701492537313

```
In [21]: y_probs = best_model.predict_proba(X_test)[:,1]
        auc_score = roc_auc_score(y_test, y_probs)
        fpr, tpr, thresholds = roc_curve(y_test, y_probs)
        plt.figure(figsize=(6,4))
        plt.plot(fpr, tpr, label=f"Logistic Regression (AUC = {auc_score:.2f})")
        plt.plot([0, 1], [0, 1], linestyle='--')
        plt.xlabel("False Positive Rate")
        plt.ylabel("True Positive Rate")
        plt.title("ROC Curve")
        plt.legend()
        plt.grid()
        plt.show()
```







```
In [27]: ct = pd.crosstab(df['Pclass'], df['Survived'], normalize='index') * 100
ct.plot(kind='bar', stacked=True, colormap='viridis', figsize=(5, 4))
plt.title("Survival Rate by Passenger Class (%)", fontsize=14)
plt.ylabel("Percentage")
plt.xlabel("Passenger Class")
plt.legend(["Did Not Survive", "Survived"])
plt.grid(axis='y')
plt.show()
```

